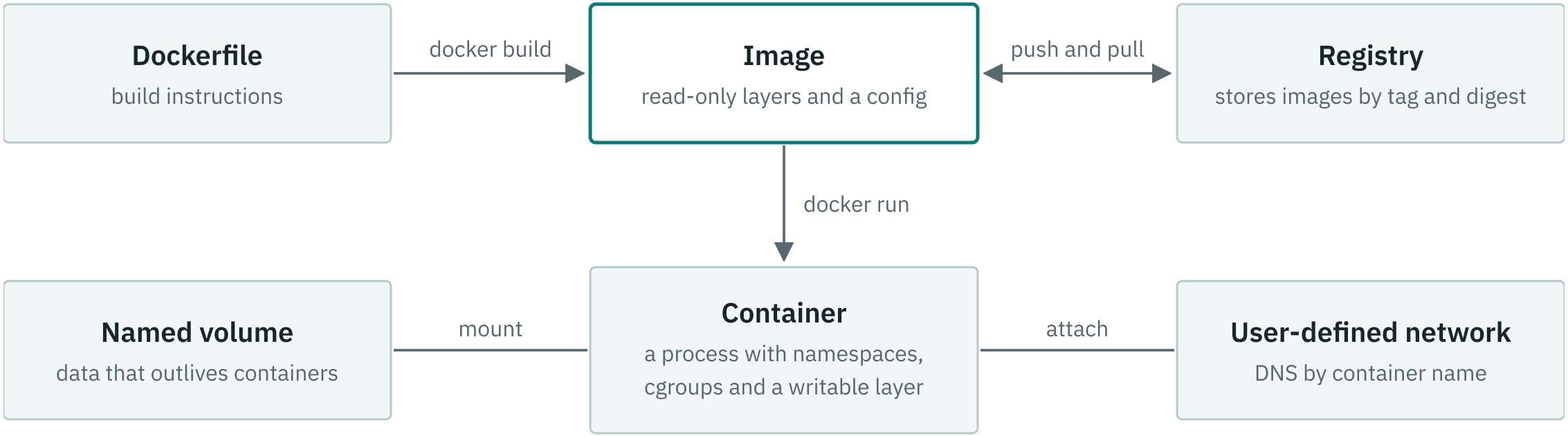
Containerization Fast Track

Day 2: Docker in depth

United Airlines · Pluralsight instructor-led training

Nick Lotz

How the Day 1 concepts connect



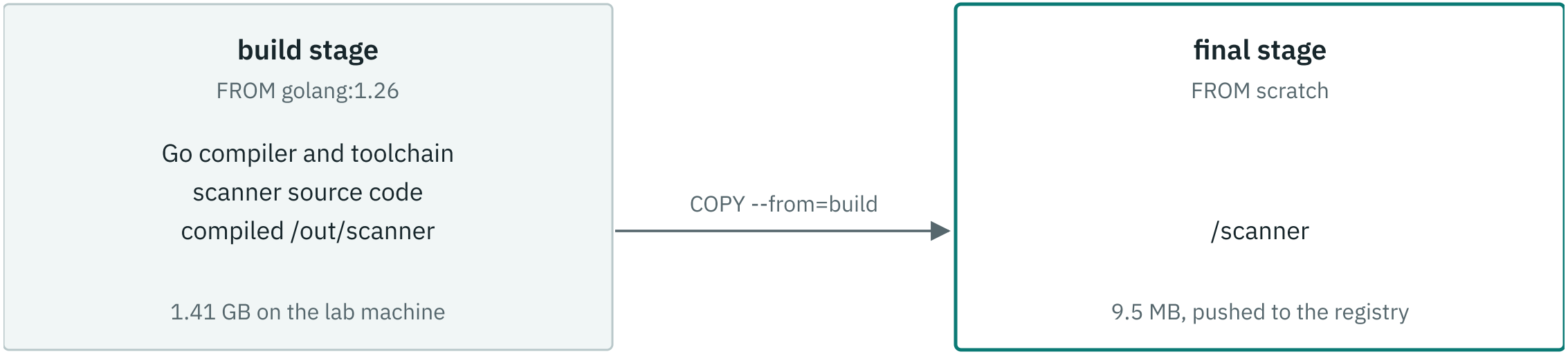
The commands on the arrows are the ones you ran in Labs 01 to 03.

Why image size matters

- Fargate pulls the whole image for each new task, because it caches no images (Amazon Web Services, n.d.).
- Every package in an image can bring vulnerabilities, even when the application never uses it.
- Shells, package managers and compilers give an attacker tools after a break-in.
- A smaller image gives a shorter vulnerability report to review.
- Registries keep every version you push, and each node stores its own copy.

A large image slows scale-out and the replacement of failed tasks.

How a multi-stage build works



Only the final stage becomes the image. The build stage stays in the build cache of the machine that ran the build.

A multi-stage Dockerfile for the scanner

```
FROM public.ecr.aws/docker/library/golang:1.26 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/scanner .

FROM scratch
COPY --from=build /out/scanner /scanner
USER 65532:65532
ENTRYPOINT ["/scanner"]
```

CGO_ENABLED=0 makes Go produce a static binary, which needs no files from the base image. USER takes numbers here, because scratch has no /etc/passwd to look up a name.

Base images compared

Image	What it contains	Size on the lab machine	High and critical findings
`python:3.13`	Debian with compilers, git and common libraries	1.62 GB	478
`python:3.13-slim`	Debian with only what Python needs to run	178 MB	51
`python:3.13-alpine`	Alpine Linux with musl libc and BusyBox	70.5 MB	5
scanner on `scratch`	One static binary and nothing else	9.5 MB	0

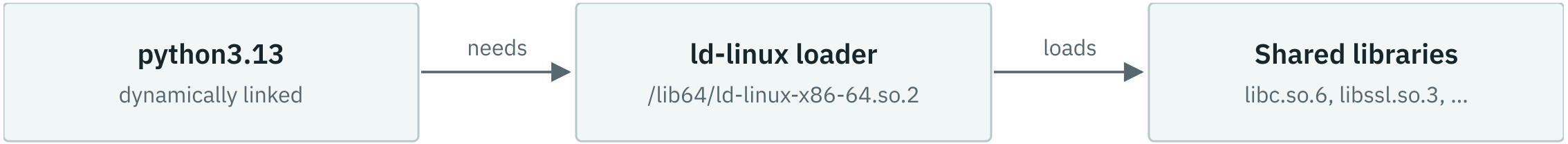
Distroless images hold only a language runtime and its libraries, with no shell or package manager (Google, n.d.). The descriptions of the python variants follow Docker's documentation (Docker, n.d.).

Alpine Linux and musl libc

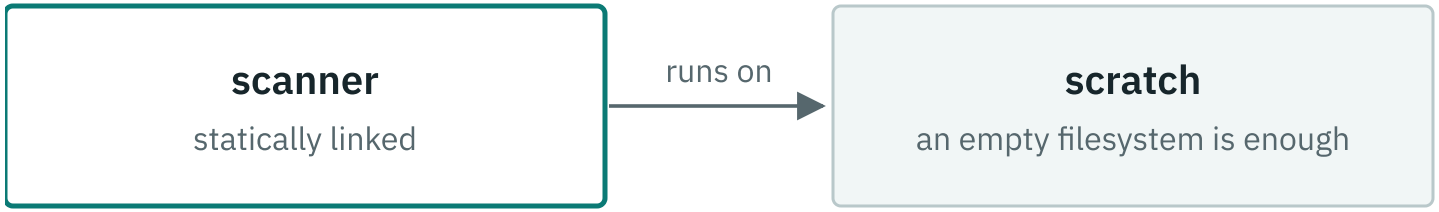
- Alpine Linux uses musl libc and BusyBox, and its base image is about 5 MB (Docker, n.d.).
- Most prebuilt Linux software, including manylinux Python wheels, is built against glibc.
- When a package has no musllinux wheel, pip builds it from source, which needs compilers in the image.
- Software that relies on glibc behaviour can fail on musl (Docker, n.d.).

For Python services, the slim Debian image avoids these problems.

Static and dynamic binaries



The base image must contain the loader and every library the binary needs.



The file and ldd commands show how a binary is linked. A static binary that calls HTTPS services also needs CA certificates copied into scratch.

BuildKit cache and secret mounts

```
# syntax=docker/dockerfile:1
FROM public.ecr.aws/docker/library/python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    --mount=type=secret,id=pipconf,target=/etc/pip.conf \
    pip install -r requirements.txt
COPY bagtrack/ bagtrack/

# Build with: docker build --secret id=pipconf,src=$HOME/.config/pip/pip.conf .
```

The cache mount keeps pip's downloads on the builder between builds. The secret file exists only for the duration of that RUN instruction (Docker, n.d.).

Measuring image size and layers

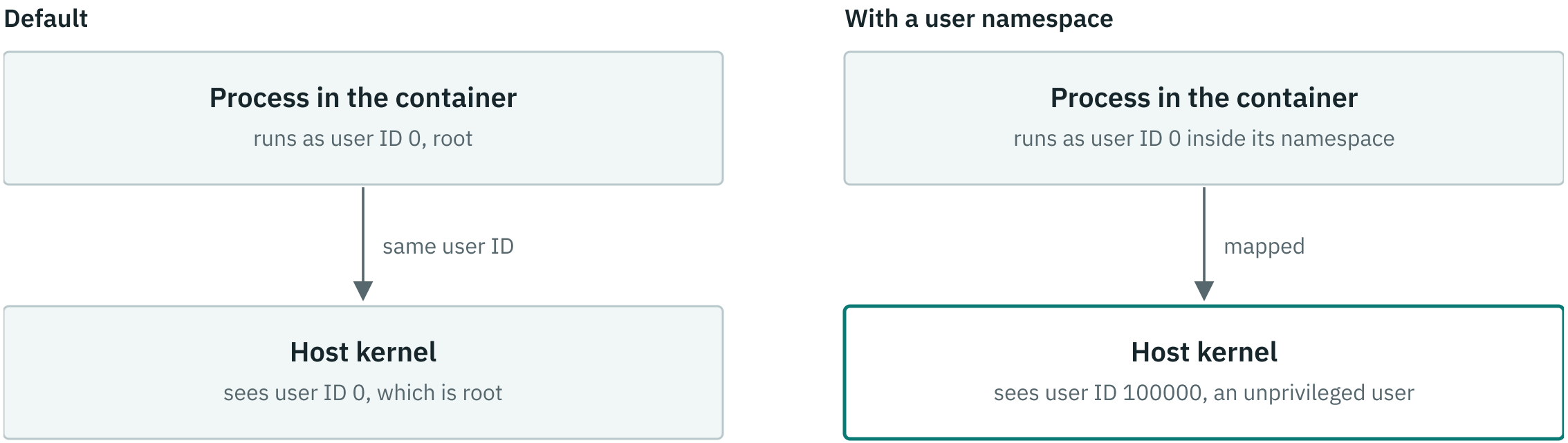
```
# Each image and its size (bag-api:1.0 is 246 MB on the lab machine)
docker image ls bag-api

# Each layer, with its size and the instruction that created it
docker image history bag-api:1.0

# The configuration and the layer digests
docker image inspect bag-api:1.0
```

A file deleted by a later instruction still takes space in the layer that added it. Clean up in the same RUN instruction, for example with `rm -rf /var/lib/apt/lists/*` after `apt-get install`.

Root in a container is root on the host



Capabilities and namespaces limit a root process, but any host file it can reach is open to root (Docker, n.d.). A user namespace maps container user IDs to unprivileged host IDs (Kerrisk, n.d.).

Access to the Docker socket is root access to the host

```
$ docker run --rm -it -v /:/host public.ecr.aws/docker/library/alpine chroot /host
# id -u
0
```

Docker's documentation warns that the docker group grants root-level privileges (Docker, n.d.). Portainer and Harness Open Source mount this socket on your lab VM, so treat their logins as root logins.

Running the process as a non-root user

```
FROM public.ecr.aws/docker/library/python:3.13-slim
RUN useradd --system --uid 10001 --no-create-home bagtrack
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY bagtrack/ bagtrack/
USER 10001
EXPOSE 8000
CMD ["uvicorn", "bagtrack.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

COPY makes root the owner of the files, so the app user can read its code but cannot change it. A numeric ID also lets Kubernetes verify that the container does not run as root.

Restricting a container at run time

`docker run` option	Effect
`--read-only`	Mounts the container's root filesystem read-only
`--tmpfs /tmp`	Adds a writable directory in memory for temporary files
`--cap-drop ALL`	Removes every Linux capability; add back only what the app needs with `--cap-add`
`--security-opt no-new-privileges`	Stops setuid programs from raising the process's privileges
`--user 10001`	Runs the process as this user, whatever the image sets

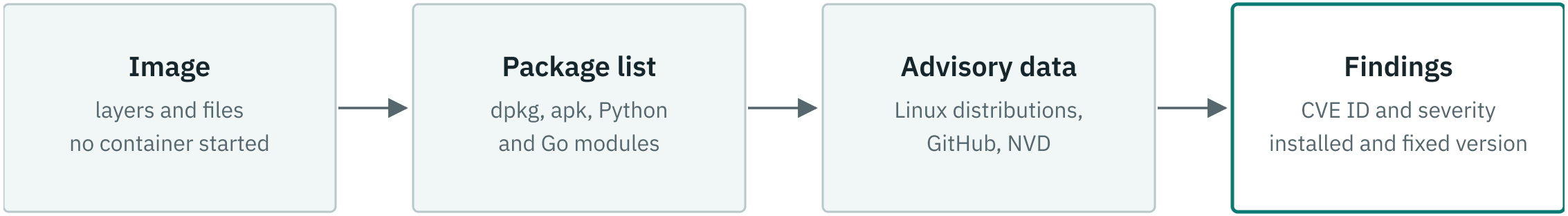
The CIS Docker Benchmark recommends a read-only root filesystem, restricted capabilities and no-new-privileges (Center for Internet Security, n.d.). Compose and Kubernetes have a matching setting for each option.

User namespaces and rootless mode

	Default	userns-remap	Rootless mode
The daemon runs as	root	root	your user
Container root maps to	root	an unprivileged ID from `/etc/subuid`	your user
Turned on with	Nothing	`"userns-remap": "default"` in `daemon.json`	`dockerd-rootless-setuptool.sh install`

Rootless mode also guards the host against vulnerabilities in the daemon and the container runtime (Docker, n.d.).

What a vulnerability scanner reports



A CVE ID names one publicly disclosed vulnerability (CVE Program, n.d.). CVSS rates its severity on a scale from 0.0 to 10.0 (FIRST, 2019).

Reading severity and fix status

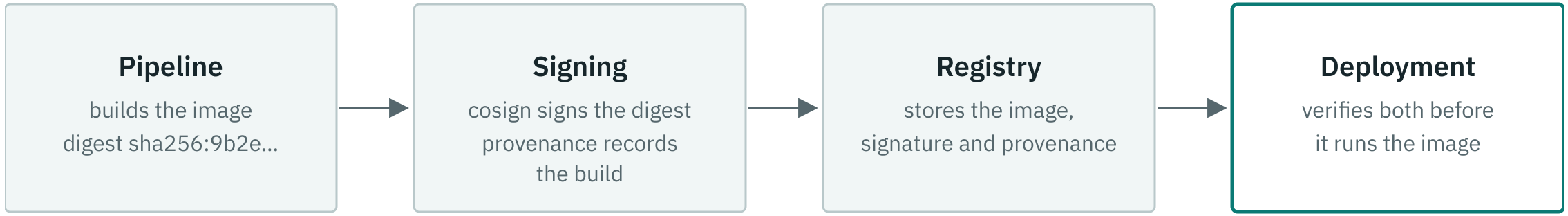
Term	Meaning	What to do
Severity	Critical 9.0–10.0, high 7.0–8.9, medium 4.0–6.9, low 0.1–3.9	Start with critical and high findings
`fixed`	A newer package version contains the fix	Update the package or the base image, then rebuild
`affected`	No fix has been released yet	Remove the package, limit exposure or accept the risk for a set time
`will_not_fix`	The distribution does not plan to fix it	Judge the risk for your use, or change the base image

The severity ranges are those of CVSS v3.1 (FIRST, 2019). Trivy's `--ignore-unfixed` option shows only findings that have a fix (Aqua Security, n.d.).

Software bills of materials

- An SBOM lists the components of an artifact, with each component's name, version and often its licence.
- CycloneDX, from OWASP, and SPDX, from the Linux Foundation, are the two common formats (CycloneDX, n.d.; SPDX, n.d.).
- SPDX is also the international standard ISO/IEC 5962:2021.
- Trivy writes both formats, for example with `trivy image --format cyclonedx bag-api:1.0`.
- When a new CVE appears, a search of stored SBOMs shows which images contain the affected package.

Signing and provenance



A signature shows who published an image (Sigstore, n.d.). Provenance records who built the image, with which process and from which inputs (SLSA, 2023).

Risk areas in NIST SP 800-190

Area	Example risks	In this course
Images	Vulnerabilities, embedded secrets, untrusted images	Scanning, BuildKit secrets, official bases
Registries	Insecure connections, stale images, weak access control	Tags, digests and the lab registry
Orchestrators	Unbounded administrative access, mixed workload sensitivity	Day 3
Containers	Runtime vulnerabilities, unbounded network access, insecure settings	Non-root, read-only, dropped capabilities
Host OS	Shared kernel, large attack surface, improper user access rights	The Docker socket, rootless mode

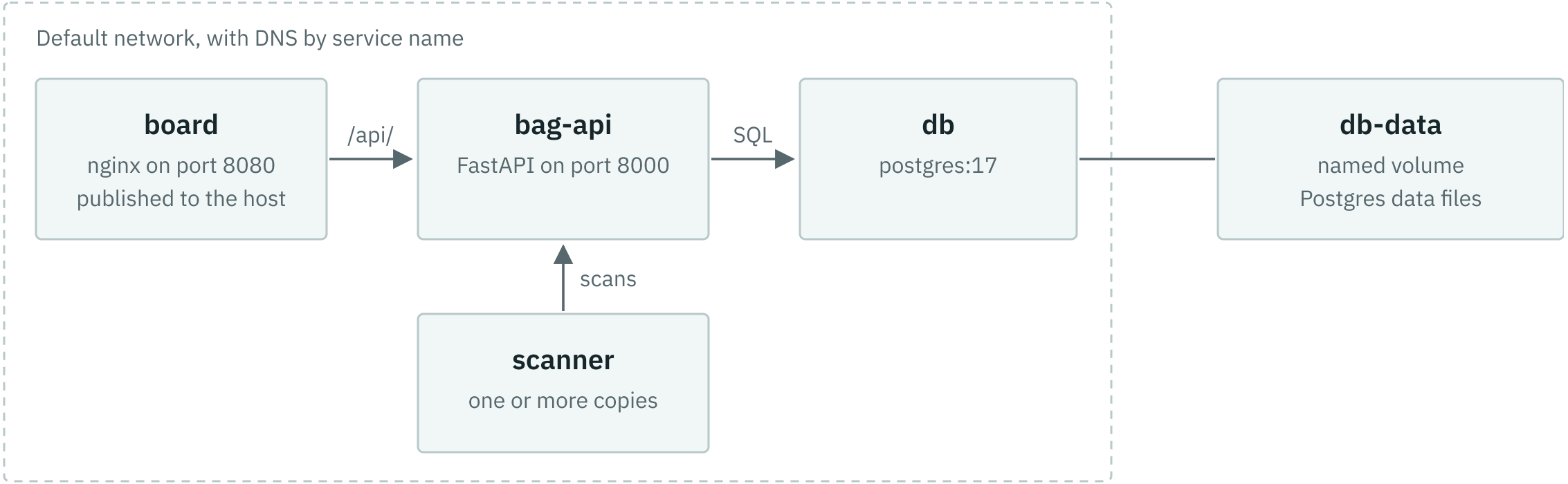
NIST groups the risks of container technologies into these five areas (Souppaya et al., 2017).

Lab 04: Smaller, safer images

`labs/04-small-secure-images.md`

- Build the scanner in two stages, ending in `scratch`
- Compare the full, slim and Alpine Python base images
- Run bag-api as non-root, read-only and with capabilities dropped
- Scan with Trivy, fix a finding and produce an SBOM
- Write a Dockerfile for gate-display and grade it with `check.sh`

The Compose model



One compose.yaml describes the services, the network and the volume. docker compose up creates them and starts the containers in dependency order (Docker, n.d.).

Start order with depends_on and healthchecks

```
services:
  db:
    image: public.ecr.aws/docker/library/postgres:17
    environment: { POSTGRES_USER: bagtrack, POSTGRES_PASSWORD: bagtrack-dev-pw, POSTGRES_DB: bagtrack }
    volumes: [ "db-data:/var/lib/postgresql/data" ]
    healthcheck:
      test: [ "CMD", "pg_isready", "-U", "bagtrack" ]
  bag-api:
    build: ./bag-api
    environment:
      DATABASE_URL: postgresql://bagtrack:bagtrack-dev-pw@db:5432/bagtrack
    depends_on:
      db: { condition: service_healthy }
volumes: { db-data: {} }
```

Compose starts bag-api only after the db healthcheck passes (Docker, n.d.). Without a condition, depends_on waits only until the db container has started.

Environment, profiles and override files

Feature	What it does
`environment`, `env_file`	Set variables inside the service's containers
`.env`	Fills `${VAR}` placeholders in compose.yaml
`profiles`	Starts a service only when its profile is enabled, for example with `--profile debug`
`compose.override.yaml`	Merged on top of compose.yaml automatically
`-f`	Merges the files you name, in the order you name them

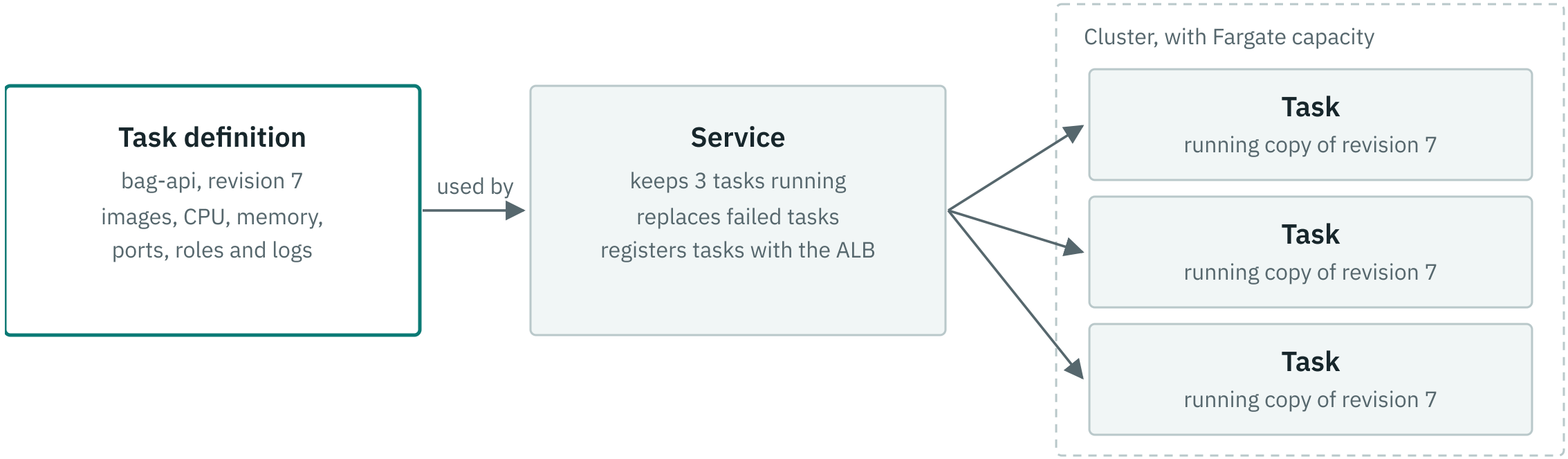
Values in .env reach a container only through environment or env_file (Docker, n.d.).

A development override with live reload

```
# compose.override.yaml, merged into compose.yaml by docker compose up
services:
  bag-api:
    volumes:
      - ./bag-api/bagtrack:/app/bagtrack
    command: [ "uvicorn", "bagtrack.main:app", "--host", "0.0.0.0",
               "--port", "8000", "--reload" ]
    environment:
      APP_VERSION: dev
```

The bind mount puts your working copy of the code in place of the image's copy. uvicorn --reload restarts the app each time a file changes.

Amazon ECS concepts



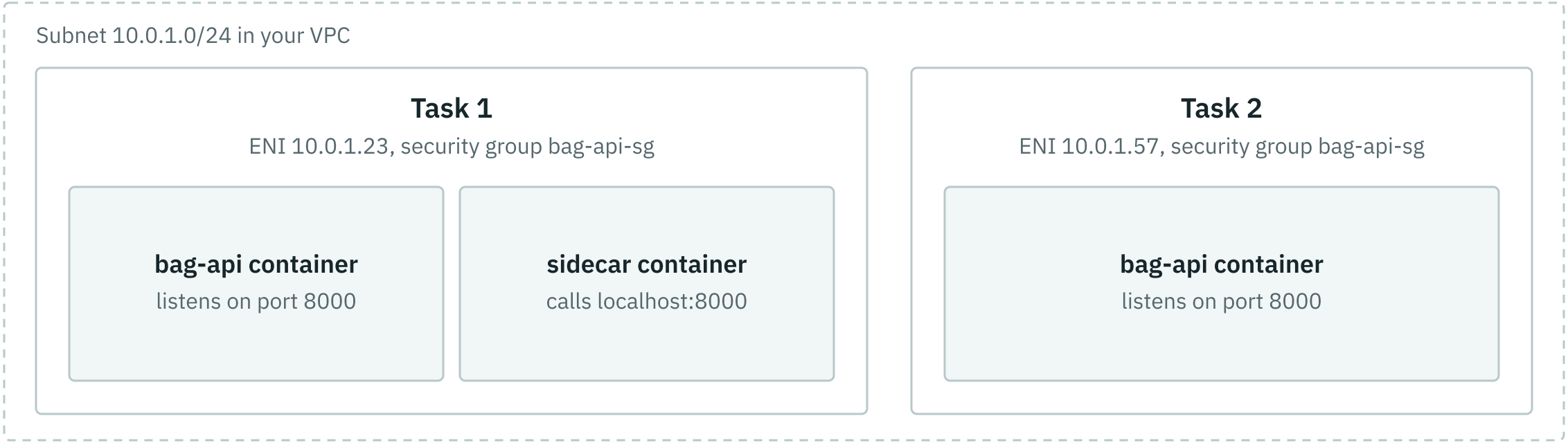
A task definition is a versioned JSON description of one or more containers (Amazon Web Services, n.d.). A task is one running copy of a task definition.

Fargate and EC2 launch types

	Fargate	EC2
Servers	AWS runs them	You run container instances, usually in an Auto Scaling group
Isolation	Each task runs on its own single-use instance (Amazon Web Services, n.d.)	Tasks share an instance and its kernel
Billing	The vCPU and memory each task requests	The instances, busy or idle
Network modes	`awsvpc` only	`awsvpc`, `bridge`, `host` or `none`
Image cache	None; each task pulls its images	Instances reuse layers they have already pulled

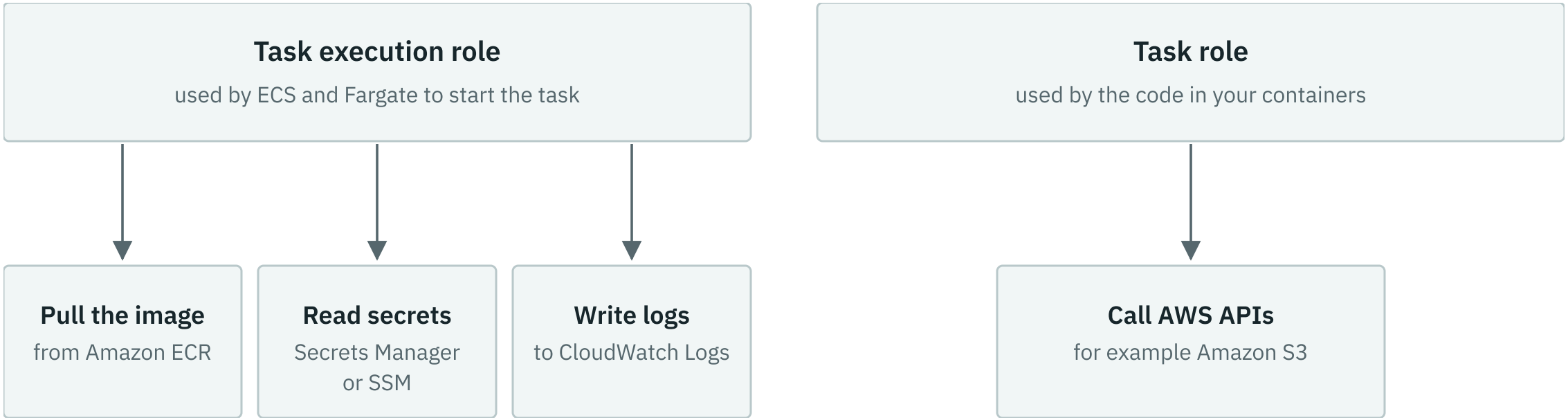
Both launch types run the same kind of task definitions and services.

Task networking with awsvpc



With awsvpc, each task gets its own elastic network interface and private IP address (Amazon Web Services, n.d.). Containers in one task share that interface and reach each other on localhost.

Task execution role and task role



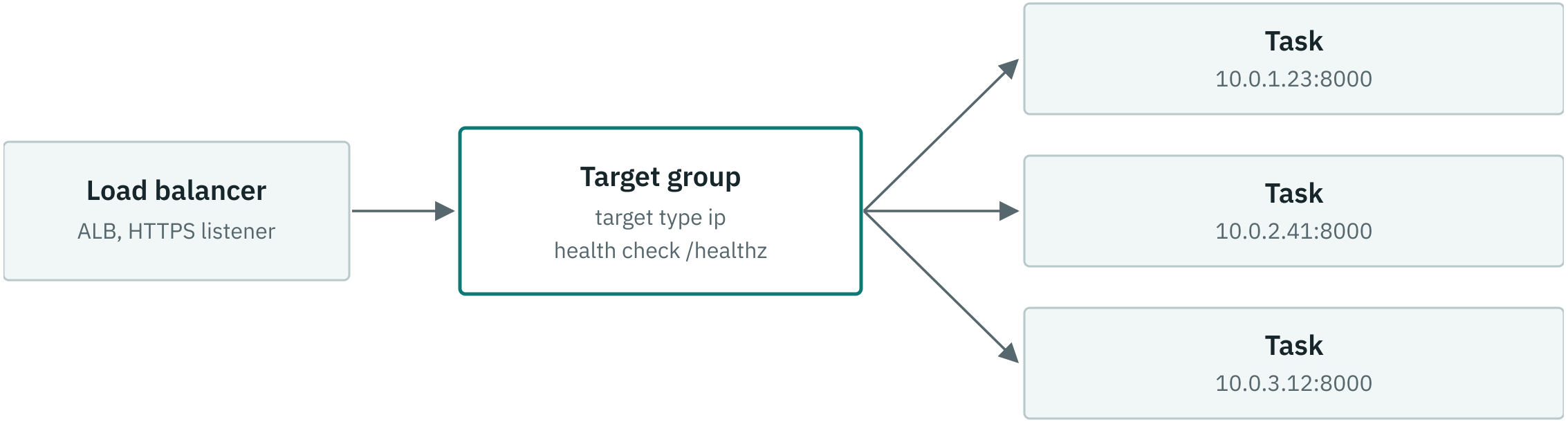
Containers receive credentials for the task role only (Amazon Web Services, n.d.). The execution role's permissions stay with ECS and Fargate.

Secrets and logs in a task definition

```
"containerDefinitions": [{
  "name": "bag-api",
  "image": "111122223333.dkr.ecr.us-east-1.amazonaws.com/bag-api:1.0.42",
  "portMappings": [{ "containerPort": 8000 }],
  "secrets": [{
    "name": "DATABASE_URL",
    "valueFrom": "arn:aws:secretsmanager:us-east-1:111122223333:secret:bagtrack-db-a1B2c3"
  }],
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": { "awslogs-group": "/ecs/bag-api", "awslogs-region": "us-east-1",
                 "awslogs-stream-prefix": "bag-api" }
  }
}]
```

ECS reads the secret with the execution role and sets it as an environment variable when the container starts (Amazon Web Services, n.d.). The awslogs driver sends the container's stdout and stderr to CloudWatch Logs.

A service behind an Application Load Balancer



With awsvpc, the target type must be ip, because each task has its own address (Amazon Web Services, n.d.). The service registers new tasks and replaces tasks that fail the health check.

Compose attributes and task-definition fields

`compose.yaml`	ECS task definition
`image`	`image`
`ports`	`portMappings`
`environment`	`environment`
Secret values	`secrets`, each with a `valueFrom` ARN
`healthcheck`	`healthCheck`
`depends_on` with a condition	`dependsOn`, for containers in the same task only
`deploy.resources.limits`	Task-level `cpu` and `memory`
`--scale scanner=3`	The service's `desiredCount`

Named volumes and Compose networks have no matching field. Fargate tasks use Amazon EFS for shared files and the VPC for networking.

Lab 05: Compose, then ECS

`labs/05-compose-and-ecs.md`

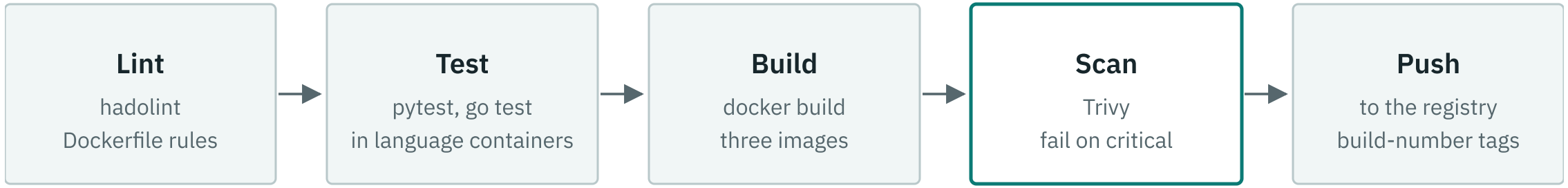
- Describe all of BagTrack in `compose.yaml`, with healthchecks and a volume
- Add a development override with live reload
- Scale the scanner and watch the board
- Write ECS task definitions for bag-api and the scanner
- Map each Compose attribute to its task-definition field

Why build images in a CI pipeline

- Every build starts from a clean checkout in the same environment, so results repeat.
- Dockerfile changes go through pull requests and review like any other code.
- Every image is linted, tested and scanned before anyone can deploy it.
- Each image tag leads back to the commit and the pipeline run that built it.
- Registry credentials stay in the CI system instead of on laptops.

Build each image once and deploy that same image to every environment (Humble and Farley, 2010).

Pipeline stages for an image



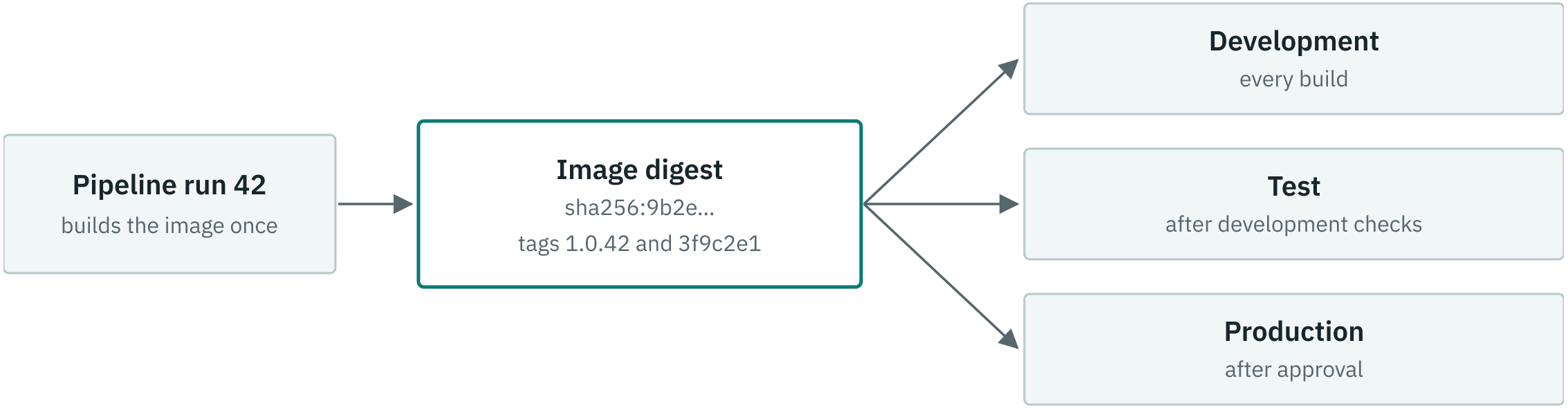
A failed stage stops the run, so later stages never start. The cheapest checks run first.

Tagging images

Tag	Example	What it tells you
Commit SHA	`bag-api:3f9c2e1`	The source commit
Version and build number	`bag-api:1.0.42`	Run 42 of the pipeline, from `1.0.${{ build.number }}`
`latest`	`bag-api:latest`	Whatever was pushed last; it moves with every push

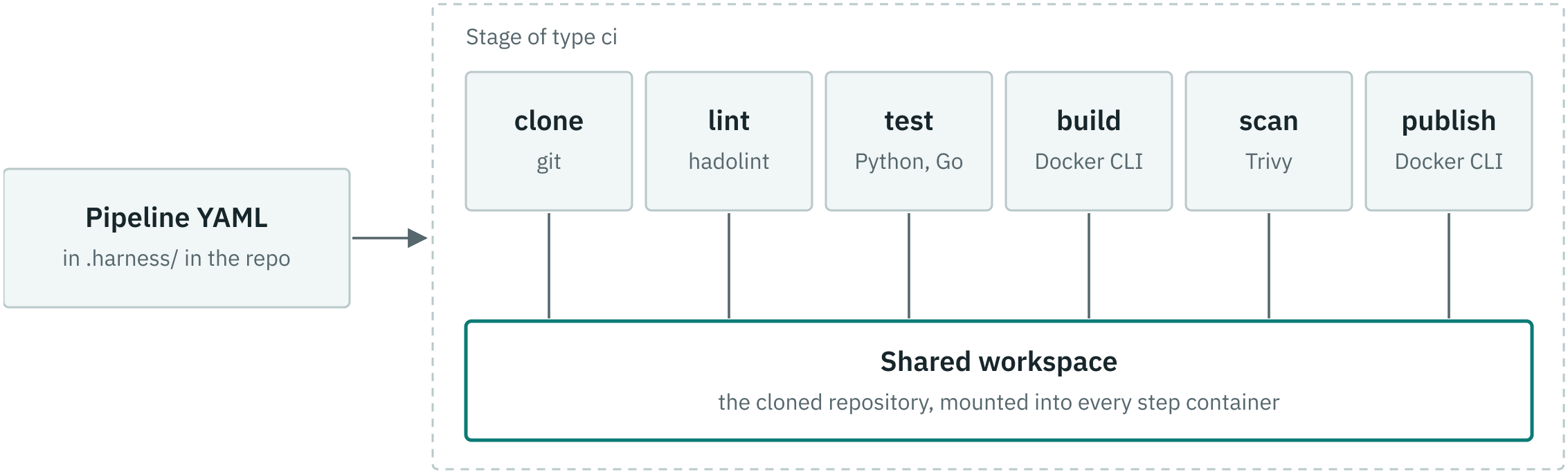
Treat every tag except latest as immutable. Amazon ECR can enforce this with tag immutability on a repository (Amazon Web Services, n.d.).

Promoting one digest through environments



Each environment runs the same digest, so production runs the image that passed the tests (Humble and Farley, 2010). Deploying by digest stops a moved tag from changing what runs.

How Harness Open Source runs a pipeline



Each step runs in its own container on the server's Docker engine (Harness, n.d.). Stages do not share files with each other.

Building images with the machine's Docker engine

```
volumes:
- name: docker-socket
  type: host
  spec:
    path: /var/run/docker.sock
steps:
- name: build
  type: run
  spec:
    container: public.ecr.aws/docker/library/docker:29-cli
    mount:
    - name: docker-socket
      path: /var/run/docker.sock
    script: |
      docker build -t localhost:5000/bag-api:1.0.${{ build.number }} ./bag-api
```

Through the socket, the build step uses the lab machine's Docker engine and its build cache. Socket access gives the pipeline full control of that engine (slide 12), so use this only on a one-person machine such as the lab VM.

A scan step that fails the pipeline

Part of the command	Effect
`trivy image --image-src docker`	Reads the image from the machine's Docker engine
`--severity CRITICAL`	Reports only critical findings
`--ignore-unfixed`	Leaves out findings that have no fixed version
`--exit-code 1`	Exits with code 1 when any finding remains (Aqua Security, n.d.)
`localhost:5000/bag-api:1.0.42`	The image that the build step tagged

The pipeline runs build, scan and publish in that order, so an image that fails the scan is never pushed. A non-zero exit code fails the run step, and the pipeline stops there.

How teams handle scan findings

- Start with a rebuild on the newest base image and an update of the affected packages.
- Record an accepted risk in `.trivyignore`, with a reason and an expiry date (Aqua Security, n.d.).
- Review accepted risks before they expire, because the pipeline fails again on that date.
- Rebuild and rescan released images on a schedule, because new CVEs appear for unchanged images.
- Agree in advance which severities fail the build and who may accept a risk.

Harness Open Source and the Harness platform

	Harness Open Source	Harness platform
Structure	Projects that hold repositories and secrets	Account, organization and project levels
Credentials	Project secrets, which appear unmasked in logs	Connectors for Git, registries and clouds such as Amazon ECR
Agents	The server runs every step itself	Delegates that run inside your network
Build steps	`run`, `plugin`, `background`; `parallel` steps run one after another; no step cache	Build and Push to ECR, test intelligence, caching
Deployment	No services, environments or approvals	Services, environments, strategies, approvals and rollback for ECS and Kubernetes

Both organise work as pipelines, stages and steps, so the ideas from Lab 06 apply to both.

Lab 06: Build, scan and publish with Harness Open Source

`labs/06-harness-ci.md`

- Start Harness Open Source, log in as admin, and create a project and a repository
- Push BagTrack and add the pipeline
- Lint, test, build, scan and publish the three images
- Update the board's base image when the scan fails
- Find the published tags in the registry

References

Amazon Web Services. (n.d.). *Amazon ECS Developer Guide*, including AWS Fargate; *Amazon ECR User Guide*. docs.aws.amazon.com

Aqua Security. (n.d.). *Trivy documentation*. trivy.dev

Center for Internet Security. (n.d.). *CIS Docker Benchmark*. cisecurity.org

CVE Program. (n.d.). *About the CVE Program*. cve.org

Docker. (n.d.). *Docker documentation*: multi-stage builds, build cache and secrets, Engine security, rootless mode, Compose; *Docker Official Images: python*. docs.docker.com

FIRST. (2019). *Common Vulnerability Scoring System v3.1: Specification document*. first.org/cvss

Google. (n.d.). *Distroless container images*. github.com/GoogleContainerTools/distroless

hadolint. (n.d.). *Haskell Dockerfile Linter*. github.com/hadolint/hadolint

Harness. (n.d.). *Harness Open Source documentation*; *Harness CI documentation*. developer.harness.io

Humble, J., and Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.

Kerrisk, M. (n.d.). *Linux manual pages*: capabilities(7), user_namespaces(7), ld.so(8). man7.org

SLSA. (2023). *Supply-chain Levels for Software Artifacts, v1.0*. slsa.dev; CycloneDX (cyclonedx.org); SPDX (spdx.dev); Sigstore (sigstore.dev)

Souppaya, M., Morello, J., and Scarfone, K. (2017). *Application Container Security Guide* (NIST SP 800-190). NIST.